API Documentation Guide

Complete REST API Reference — All Microservices

Real-Time Order Processing System

Version: 2.0 | **Date:** March 2026 **Base URL:** `http://localhost:8088` (via API Gateway) **Authentication:** JWT Bearer Token **Total Endpoints:** 43

Table of Contents

1. API Overview Matrix
2. Authentication & Headers
3. API Gateway (Routes & Fallbacks)
4. Auth Service APIs
5. User Service APIs
6. Product Service APIs
7. Order Service APIs
8. Inventory Service APIs
9. Payment Service APIs
10. Notification Service APIs
11. Error Handling Strategy
12. API Design Patterns
13. Interview Questions

---

1. API Overview Matrix

```
+────────────────+──────+──────────────────────────────+──────+──────+──────────+
| Service        | Port | Base Path                    | Auth | RBAC | Count    |
+────────────────+──────+──────────────────────────────+──────+──────+──────────+
| Auth Service   | 8081 | /api/auth/*                  |  No  |  No  |  3       |
| User Service   | 8082 | /api/users/*  /admin/users/* |  Yes |  Yes |  9       |
| Product Svc    | 8083 | /api/products/*  /admin/prod.* |  Yes |  Yes |  8       |
| Order Service  | 8084 | /api/orders/*  /admin/orders/* |  Yes |  Yes |  8       |
| Inventory Svc  | 8085 | /api/inventory/* /admin/inv.* |  Yes |  Yes |  7       |
| Payment Svc    | 8086 | /api/payments/*              |  Yes |  No  |  1       |
| Notification   | 8087 | /api/notifications/*         |  Yes |  Yes |  4       |
| API Gateway    | 8088 | /fallback/*                  |  No  |  No  |  2+      |
+────────────────+──────+──────────────────────────────+──────+──────+──────────+
|                                                        TOTAL      |  43      |
+──────────────────────────────────────────────────────────────────────+──────────+
```

---

2. Authentication & Headers

2.1 JWT Bearer Token

```
ALL requests (except /api/auth/*) require:
  Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
```

```
HOW TO GET A TOKEN:
  1. POST /api/auth/signup  → Register
  2. POST /api/auth/login   → Get JWT token
  3. Use token in all subsequent requests
```

2.2 Gateway-Injected Headers

```
After JWT validation, the API Gateway injects:
  X-User-Name: john           (extracted from JWT sub claim)
  X-User-Role: ROLE_USER      (extracted from JWT role claim)

Downstream services READ these headers — they never parse JWT themselves.
```

2.3 Request Flow

```
Frontend → API Gateway (:8088) → JwtAuthenticationFilter → Service
                                    │
                                    ├─ Open endpoint? → skip JWT → forward
                                    ├─ Valid JWT? → add X-User-Name, X-User-Role → forward
                                    └─ Invalid JWT? → 401 Unauthorized
```

---

3. API Gateway

The Gateway itself does not expose business APIs — it routes requests to downstream services. It provides fallback endpoints when services are unavailable.

🔹 Auth Service Fallback

Method: ANY **URL:** `/fallback/auth` **Authentication Required:** No

1️⃣ What

Returns a friendly error response when Auth Service is unavailable (circuit breaker OPEN).

2️⃣ Why

Instead of the user seeing a raw timeout or connection error, they get a structured JSON response explaining the issue.

3️⃣ When

Called automatically by the circuit breaker when Auth Service failure rate exceeds the threshold (default 50%) within the sliding window.

4️⃣ How

Gateway route has `fallbackUri: forward:/fallback/auth`. When circuit opens → Gateway redirects to `FallbackController.authServiceFallback()` → returns 503 with details.

Response Example (503)

```
{
  "error": "Auth Service is currently unavailable",
  "exception": "io.netty.channel.ConnectTimeoutException",
  "details": "connection timed out",
  "message": "Please try again later. The authentication service is experiencing issues.",
  "timestamp": "2026-03-03T15:30:00",
  "status": 503
}
```

🔹 Generic Service Fallback

Method: ANY **URL:** `/fallback/generic` **Authentication Required:** No

Same pattern as auth fallback — used for all other services (Product, Order, Inventory, etc.).

---

4. Auth Service APIs

Base URL: `/api/auth` | **Port:** 8081 | **Database:** auth_db **Authentication:** None (public endpoints)

🔹 User Registration (Signup)

Method: `POST` **URL:** `/api/auth/signup` **Authentication Required:** No **Roles Allowed:** N/A (public)

1️⃣ What

Registers a new user in the system. Creates a `UserCredential` entity with BCrypt-hashed password and assigns a role (USER or ADMIN).

2️⃣ Why

This is the entry point for new users. Without it, no one can authenticate. It establishes the user's identity (username, email) and security credentials (hashed password, role) used by all other services.

3️⃣ When

- User clicks "Register" on the frontend
- Admin creates a new admin account
- System bootstrap (creating the first admin user)

4️⃣ How

```
1. AuthController.signup(@Valid @RequestBody SignupRequest)
2. Validation: @NotBlank username, @Email email, @Size(min=6) password
3. AuthService.signup():
   a. Check username uniqueness → existsByUsername()
   b. Check email uniqueness → existsByEmail()
   c. Determine role (default USER, "ADMIN" if specified)
   d. Hash password: passwordEncoder.encode(password) → BCrypt 12 rounds
   e. Save UserCredential to auth_db
4. Return 201 Created with AuthResponse (no token — user must login)
```

Request Example

```
{
  "username": "john_doe",
  "email": "john@example.com",
  "password": "SecurePass123!",
  "role": "USER"
}
```

Response Example (201 Created)

```
{
  "token": null,
  "username": "john_doe",
  "role": "USER",
  "message": "User registered successfully"
}
```

Error Responses

Status	Scenario	Body
400	Username already exists	`{"message": "Username already exists: john_doe", "status": 400}`
400	Email already exists	`{"message": "Email already exists: john@example.com", "status": 400}`
400	Validation failure	`{"errors": {"password": "size must be between 6 and 2147483647"}}`

🔹 User Login

Method: `POST` **URL:** `/api/auth/login` **Authentication Required:** No **Roles Allowed:** N/A (public)

1️⃣ What

Authenticates a user and returns a JWT token. This token is used for all subsequent API calls.

2️⃣ Why

Stateless authentication — the returned JWT contains username and role. No server-side sessions needed. The token is self-contained and can be validated by any service instance.

3️⃣ When

- User clicks "Login" on the frontend
- Token expires and user must re-authenticate
- Automated testing setup

4️⃣ How

```
1. AuthController.login(@Valid @RequestBody AuthRequest)
2. AuthService.login():
   a. findByUsername() → throws "User not found" if missing
   b. passwordEncoder.matches(provided, stored) → BCrypt comparison
   c. If mismatch → throws "Invalid credentials"
```

```
   d. jwtUtil.generateToken(username, role) → JWT with 24h expiry
3. Return 200 OK with AuthResponse containing JWT token
```

Request Example

```
{
  "username": "john_doe",
  "password": "SecurePass123!"
}
```

Response Example (200 OK)

```
{
  "token":
"eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJqb2huX2RvZSIsInJvbGUiOiJVU0VSIiwiaWF0IjoxNzE0Njk1NDIzLCJleHAiO
  "username": "john_doe",
  "role": "USER",
  "message": "Login successful"
}
```

Error Responses

Status	Scenario	Body
400	User not found	`{"message": "User not found: john_doe", "status": 400}`
400	Wrong password	`{"message": "Invalid credentials", "status": 400}`

🔹 Token Validation

Method: `GET` **URL:** `/api/auth/validate?token={jwt}` **Authentication Required:** No **Roles Allowed:** N/A (public)

1️⃣ What

Validates whether a JWT token is still valid (not expired, correctly signed).

2️⃣ Why

Used by the API Gateway and frontend to check token validity without requiring a full login. Useful for session persistence checks on page reload.

3️⃣ When

- Frontend app startup (check if stored token is still valid)
- Gateway filter could call this for token validation (alternative to local validation)
- Health check / debugging

4️⃣ How

```
1. AuthController.validateToken(@RequestParam token)
2. AuthService.validateToken(token)
3. JwtUtil.isTokenValid(token):
```

```
   a. Parse token with shared secret key
   b. Check expiry: claims.getExpiration().after(new Date())
   c. Verify HMAC-SHA256 signature
4. Return 200 OK with {valid: true/false}
```

Request Example

```
GET /api/auth/validate?token=eyJhbGciOiJIUzI1NiJ9...
```

Response Example (200 OK)

```
{
  "valid": true,
  "message": "Token is valid"
}
```

---

5. User Service APIs

Base URL: `/api/users` (user) | `/admin/users` (admin) **Port:** 8082 | **Database:** user_db **Kafka:** Produces user-events, audit-logs

---

🔹 Create User Profile

Method: `POST` **URL:** `/api/users` **Authentication Required:** Yes **Roles Allowed:** USER, ADMIN

1️⃣ What

Creates a new user profile with personal details (full name, phone, address). Separate from Auth's signup — Auth handles credentials, User handles profile data.

2️⃣ Why

Separation of concerns: Auth Service manages authentication (username/password/JWT). User Service manages profile data (name, phone, address). Different databases, different schemas, different scaling needs.

3️⃣ When

- After successful signup, frontend calls this to create the profile
- Admin creates a profile for a user

4️⃣ How

```
1. UserController.createProfile(@Valid @RequestBody UserProfileDto)
2. UserService.createProfile() → save to user_db
3. Return 201 Created with UserProfileDto
```

Request Example

```
{
  "username": "john_doe",
  "fullName": "John Doe",
```

```
  "email": "john@example.com",
  "phone": "+1-555-0123",
  "address": "123 Main St, Springfield"
}
```

🔹 Get Profile by ID

Method: `GET` **URL:** `/api/users/{id}` **Authentication Required:** Yes

1️⃣ What

Retrieves a user profile by numeric ID.

Request Example

```
GET /api/users/1
```

Response Example (200 OK)

```
{
  "id": 1,
  "username": "john_doe",
  "fullName": "John Doe",
  "email": "john@example.com",
  "phone": "+1-555-0123",
  "address": "123 Main St"
}
```

🔹 Get Profile by Username

Method: `GET` **URL:** `/api/users/username/{username}` **Authentication Required:** Yes

1️⃣ What

Retrieves a user profile by username. Used more often than ID lookup since Gateway provides X-User-Name header (not user ID).

🔹 Get All Profiles

Method: `GET` **URL:** `/api/users` **Authentication Required:** Yes

🔹 Update Profile

Method: `PUT` **URL:** `/api/users/{id}` **Authentication Required:** Yes

Request Example

```
{
  "fullName": "John A. Doe",
  "phone": "+1-555-9999",
```

```
  "address": "456 Oak Ave"
}
```

🔹 [ADMIN] Get All Users

Method: `GET` **URL:** `/admin/users` **Authentication Required:** Yes **Roles Allowed:** ADMIN only (`@PreAuthorize("hasRole('ADMIN')")`)

1️⃣ What

Returns all user profiles. Admin dashboard uses this to manage users.

2️⃣ Why

Admin needs visibility into all registered users for management, support, and compliance.

🔹 [ADMIN] Block User

Method: `PUT` **URL:** `/admin/users/{id}/block` **Authentication Required:** Yes **Roles Allowed:** ADMIN only

1️⃣ What

Blocks a user account, preventing them from using the system.

2️⃣ Why

Security/compliance — suspicious activity, fraud detection, terms of service violation.

4️⃣ How

```
1. AdminUserController.blockUser(id, X-User-Name header)
2. UserService.blockUser(id) → update user status in user_db
3. Publish AuditLogEvent to Kafka ("audit-logs" topic):
   {actor: "admin", action: "BLOCK", resourceType: "USER", resourceId: "42"}
4. Publish UserEvent to Kafka ("user-events" topic):
   {userId: 42, username: "john", type: "BLOCKED"}
5. Return 204 No Content
```

🔹 [ADMIN] Unblock User

Method: `PUT` **URL:** `/admin/users/{id}/unblock` **Authentication Required:** Yes **Roles Allowed:** ADMIN only

Same pattern as block — publishes UNBLOCKED events.

🔹 [ADMIN] Delete User

Method: `DELETE` **URL:** `/admin/users/{id}` **Authentication Required:** Yes **Roles Allowed:** ADMIN only

Publishes DELETED audit log and user event to Kafka. Returns 204 No Content.

---

6. Product Service APIs

Base URL: `/api/products` (user) | `/admin/products` (admin) **Port:** 8083 | **Database:** product_db **Redis:** @Cacheable (product::id, TTL 10min), @CacheEvict (products list, TTL 5min) **Kafka:** Produces product-created, product-events, audit-logs

---

🔹 Get Product by ID

Method: `GET` **URL:** `/api/products/{id}` **Authentication Required:** Yes **Roles Allowed:** USER, ADMIN

1️⃣ What

Retrieves a single product by ID. **Cached in Redis** — first call hits MySQL, subsequent calls served from Redis (sub-millisecond).

2️⃣ Why

Product catalog is read-heavy (browse → view → buy). Caching reduces database load by ~90%.

3️⃣ When

- User clicks on a product in the catalog
- Order Service validates product exists before order placement
- Frontend product detail page

4️⃣ How

```
1. ProductController.getProduct(id)
2. ProductService.getProductById(id) — @Cacheable(value="product", key="#id")
   a. AOP proxy intercepts method call
   b. Check Redis: GET product::1
   c. Cache HIT → return cached ProductDto (0.5ms)
   d. Cache MISS → query MySQL → serialize to Redis → return (5ms)
3. Cache entry: key="product::1" value="{@class:...ProductDto, id:1, name:Laptop}"
   TTL: 10 minutes
```

Request Example

```
GET /api/products/1
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
```

Response Example (200 OK)

```
{
  "id": 1,
  "name": "MacBook Pro 16-inch",
  "description": "Apple M3 Pro chip, 18GB RAM",
  "price": 2499.99,
  "category": "Electronics",
  "imageUrl": "https://cdn.example.com/macbook.jpg"
}
```

🔹 Get All Products

Method: `GET` **URL:** `/api/products` **Authentication Required:** Yes

4️⃣ How

`@Cacheable(value="products")` — entire list cached with 5-minute TTL (shorter than individual products because list changes more frequently).

🔹 Create Product (User Path)

Method: `POST` **URL:** `/api/products` **Authentication Required:** Yes

Request Example

```
{
  "name": "MacBook Pro 16-inch",
  "description": "Apple M3 Pro chip, 18GB RAM",
  "price": 2499.99,
  "category": "Electronics",
  "imageUrl": "https://cdn.example.com/macbook.jpg"
}
```

Response (201 Created) — same as GET response with generated ID.

🔹 Update Product (User Path)

Method: `PUT` **URL:** `/api/products/{id}` **Authentication Required:** Yes

🔹 Delete Product (User Path)

Method: `DELETE` **URL:** `/api/products/{id}` **Authentication Required:** Yes **Response:** 204 No Content

---

🔹 [ADMIN] Create Product

Method: `POST` **URL:** `/admin/products` **Authentication Required:** Yes **Roles Allowed:** ADMIN only (`@PreAuthorize("hasRole('ADMIN')")`)

1️⃣ What

Creates a new product in the catalog. Additionally publishes Kafka events (product-created, audit-log) asynchronously for downstream processing.

2️⃣ Why

Admin path includes Kafka event publishing and audit logging. The product-created event triggers Inventory Service to initialize stock (quantity=0) for the new product.

3️⃣ When

- Admin adds a new product to the catalog via admin dashboard

4️⃣ How

```
1. AdminProductController.createProduct(@Valid ProductDto, X-User-Name)
2. @PreAuthorize("hasRole('ADMIN')") — checks X-User-Role header
```

```
3. ProductService.createProduct():
   a. @CacheEvict(value="products", allEntries=true) — clear list cache
   b. Save Product entity to product_db
4. CompletableFuture.runAsync(() → {  // Fire-and-forget, NEVER blocks HTTP
   a. Publish AuditLogEvent to "audit-logs" topic
   b. Publish ProductEvent to "product-events" topic
   })
5. Return 201 Created
```

Kafka Events Published

```
// Topic: audit-logs
{
  "eventId": "uuid-abc",
  "actor": "admin",
  "action": "CREATE",
  "resourceType": "PRODUCT",
  "resourceId": "42",
  "details": "Created product: MacBook Pro",
  "timestamp": "2026-03-03T15:30:00"
}

// Topic: product-events
{
  "eventId": "uuid-def",
  "productId": 42,
  "type": "CREATED",
  "name": "MacBook Pro",
  "price": 2499.99
}
```

🔹 [ADMIN] Update Product

Method: `PUT` **URL:** `/admin/products/{id}` **Authentication Required:** Yes **Roles Allowed:** ADMIN only

Same pattern — updates product + publishes audit + product-events.

🔹 [ADMIN] Delete Product

Method: `DELETE` **URL:** `/admin/products/{id}` **Authentication Required:** Yes **Roles Allowed:** ADMIN only **Response:** 204 No Content + Kafka events (DELETED)

---

7. Order Service APIs

Base URL: `/api/orders` (user) | `/admin/orders` (admin) **Port:** 8084 | **Database:** order_db **Kafka Producer:** order-created, order-cancelled, audit-logs **Kafka Consumer:** payment-success, payment-failure, inventory-updated

---

🔹 Place Order

Method: `POST` **URL:** `/api/orders` **Authentication Required:** Yes **Roles Allowed:** USER, ADMIN

1️⃣ What

Places a new order for a product. This is the **Saga Initiator** — it triggers the entire asynchronous order processing chain (Inventory → Payment → Notification).

2️⃣ Why

This is the core business operation. The order is saved with status PENDING and the actual processing (stock check, payment) happens asynchronously via Kafka.

3️⃣ When

- User clicks "Buy Now" or "Place Order" on the frontend
- After adding items to cart and confirming checkout

4️⃣ How

```
1. OrderController.placeOrder(@Valid OrderRequest, X-User-Name)
2. Validation: @NotNull productId, @Min(1) quantity, @NotNull unitPrice
3. OrderService.placeOrder():
   a. Calculate totalPrice = unitPrice × quantity
   b. Create Order entity (status = PENDING)
   c. Save to order_db
   d. Build OrderEvent with UUID eventId
   e. Publish to Kafka topic "order-created" (key = productId)
   f. Return OrderResponse immediately (user doesn't wait for payment)

AFTER PUBLISH (async, user already got 201):
   g. Inventory Service consumes → reserves stock → publishes inventory-updated
   h. Payment Service consumes → processes payment → publishes payment-success
   i. OrderStatusConsumer receives → updates order status to CONFIRMED
   j. Notification Service sends notification
```

Request Example

```json
{
  "productId": 1,
  "productName": "MacBook Pro",
  "quantity": 2,
  "unitPrice": 2499.99
}
```

Response Example (201 Created)

```json
{
  "id": 101,
  "userId": 1,
  "productId": 1,
  "productName": "MacBook Pro",
  "quantity": 2,
  "totalPrice": 4999.98,
  "status": "PENDING",
```

```
  "createdAt": "2026-03-03T15:30:00"
}
```

Validation Errors (400)

```
{
  "errors": {
    "productId": "Product ID is required",
    "quantity": "Quantity must be at least 1"
  },
  "status": 400,
  "timestamp": "2026-03-03T15:30:00"
}
```

🔹 Get Order by ID

Method: `GET` **URL:** `/api/orders/{id}` **Authentication Required:** Yes

Response Example (200 OK)

```
{
  "id": 101,
  "userId": 1,
  "productId": 1,
  "productName": "MacBook Pro",
  "quantity": 2,
  "totalPrice": 4999.98,
  "status": "CONFIRMED",
  "createdAt": "2026-03-03T15:30:00"
}
```

Order Status Values: `PENDING → INVENTORY_RESERVED → CONFIRMED | CANCELLED | PAYMENT_FAILED`

🔹 Get Orders by User

Method: `GET` **URL:** `/api/orders/user/{userId}` **Authentication Required:** Yes

Returns all orders for a specific user (order history).

🔹 Get All Orders

Method: `GET` **URL:** `/api/orders` **Authentication Required:** Yes

🔹 Cancel Order

Method: `PUT` **URL:** `/api/orders/{id}/cancel` **Authentication Required:** Yes

1️⃣ What

Cancels a pending or confirmed order. Publishes a cancellation event to Kafka so Inventory Service can release the reserved stock.

2️⃣ Why

Users may change their mind. Cancellation must trigger stock restoration (Saga compensation).

4️⃣ How

```
1. OrderService.cancelOrder(orderId, userId)
2. Verify order exists and belongs to user
3. Update status to CANCELLED
4. Publish OrderEvent to "order-cancelled" topic
5. Inventory Service consumes → restores reserved stock
```

🔹 [ADMIN] Get All Orders

Method: `GET` **URL:** `/admin/orders` **Authentication Required:** Yes **Roles Allowed:** ADMIN only

---

🔹 [ADMIN] Update Order Status

Method: `PUT` **URL:** `/admin/orders/{id}/status?status={newStatus}` **Authentication Required:** Yes **Roles Allowed:** ADMIN only

1️⃣ What

Manually updates an order's status. Admin override for stuck or disputed orders.

2️⃣ Why

If the Kafka saga stalls (e.g., consumer crash), admin can manually move orders through the pipeline.

Request Example

```
PUT /admin/orders/101/status?status=CONFIRMED
X-User-Name: admin
```

Valid Status Values

`PENDING`, `INVENTORY_RESERVED`, `CONFIRMED`, `CANCELLED`, `PAYMENT_FAILED`

🔹 [ADMIN] Cancel Order

Method: `DELETE` **URL:** `/admin/orders/{id}` **Authentication Required:** Yes **Roles Allowed:** ADMIN only

Admin cancellation also publishes an `OrderEvent` with status CANCELLED to Kafka for inventory rollback.

---

8. Inventory Service APIs

Base URL: `/api/inventory` (user) | `/admin/inventory` (admin) **Port:** 8085 | **Database:** inventory_db **Redis:** Distributed lock (SETNX, TTL 30s) **Kafka Consumer:** order-created, order-cancelled, product-created **Kafka Producer:** inventory-updated

---

🔹 Get Inventory by Product ID

Method: `GET` **URL:** `/api/inventory/{productId}` **Authentication Required:** Yes

Response Example (200 OK)

```
{
  "id": 1,
  "productId": 42,
  "quantity": 100,
  "reservedQuantity": 15,
  "availableQuantity": 85,
  "version": 3,
  "createdAt": "2026-03-01T10:00:00",
  "updatedAt": "2026-03-03T15:30:00"
}
```

🔹 Add Inventory

Method: `POST` **URL:** `/api/inventory` **Authentication Required:** Yes

1️⃣ What

Adds or initializes inventory for a product. If inventory already exists, adds to the existing quantity.

Request Example

```
{
  "productId": 42,
  "quantity": 100
}
```

🔹 Check Stock Availability

Method: `GET` **URL:** `/api/inventory/check/{productId}` **Authentication Required:** Yes

1️⃣ What

Returns a detailed stock availability report for a product, including total, reserved, available quantities and an `inStock` boolean.

2️⃣ Why

Frontend uses this before allowing "Add to Cart" — if inStock is false, show "Out of Stock" badge.

Response Example (200 OK)

```
{
  "productId": 42,
  "totalQuantity": 100,
  "reservedQuantity": 15,
```

```
  "availableQuantity": 85,
  "inStock": true
}
```

🔹 [ADMIN] Get All Inventory

Method: `GET` **URL:** `/admin/inventory` **Authentication Required:** Yes **Roles Allowed:** ADMIN only (class-level `@PreAuthorize`)

Returns all inventory records for admin dashboard stock monitoring.

🔹 [ADMIN] Initialize Inventory

Method: `POST` **URL:** `/admin/inventory` **Authentication Required:** Yes **Roles Allowed:** ADMIN only

1️⃣ What

Manually initializes inventory for a product. Used when the automatic `product-created` Kafka event was missed.

Request Example

```
{
  "productId": 42,
  "initialStock": 100
}
```

🔹 [ADMIN] Adjust Inventory

Method: `PUT` **URL:** `/admin/inventory/{productId}/adjust` **Authentication Required:** Yes **Roles Allowed:** ADMIN only

1️⃣ What

Sets the total inventory quantity to a specific value. Used for returns, damaged goods, warehouse restocking.

Request Example

```
{
  "newQuantity": 150
}
```

🔹 [ADMIN] Get Inventory Details

Method: `GET` **URL:** `/admin/inventory/{productId}` **Authentication Required:** Yes **Roles Allowed:** ADMIN only

Enhanced version of the user endpoint — includes version (optimistic lock counter), createdAt, and updatedAt timestamps.

Response Example (200 OK)

```
{
  "productId": 42,
  "totalQuantity": 100,
  "reservedQuantity": 15,
  "availableQuantity": 85,
  "version": 3,
  "createdAt": "2026-03-01T10:00:00",
  "updatedAt": "2026-03-03T15:30:00",
  "inStock": true
}
```

---

9. Payment Service APIs

Base URL: `/api/payments` | **Port:** 8086 | **Database:** payment_db **Kafka Consumer:** inventory-updated **Kafka Producer:** payment-success, payment-failure

---

🔹 Get Payment by Order ID

Method: `GET` **URL:** `/api/payments/order/{orderId}` **Authentication Required:** Yes

1️⃣ What

Retrieves the payment record associated with a specific order.

2️⃣ Why

Users and admins need to verify payment status for an order (confirmation, refund disputes, receipt generation).

3️⃣ When

- User checks "Order Details" page (shows payment status alongside order status)
- Admin investigates payment disputes

4️⃣ How

```
1. PaymentController.getPaymentByOrderId(orderId)
2. PaymentRepository.findByOrderId(orderId)
3. If not found → RuntimeException → 400 "Payment not found for orderId: X"
```

Response Example (200 OK)

```
{
  "id": 1,
  "orderId": 101,
  "amount": 4999.98,
  "status": "SUCCESS",
  "transactionId": "TXN-uuid-abc123",
  "createdAt": "2026-03-03T15:30:05"
}
```

Payment Status Values: `SUCCESS` , `FAILED`

WHY only 1 REST endpoint? Payment Service is primarily event-driven. It consumes `inventory-updated` events from Kafka and processes payments automatically. The REST endpoint is for querying payment records, not initiating payments.

---

10. Notification Service APIs

Base URL: `/api/notifications` (user) | `/admin/audit-logs` (admin) **Port:** 8087 | **Database:** notification_db
Kafka Consumer: payment-success, payment-failure, audit-logs

🔹 Get Notifications by User

Method: `GET` **URL:** `/api/notifications/user/{userId}` **Authentication Required:** Yes

1️⃣ What

Retrieves all notifications for a user, ordered by most recent first.

Response Example (200 OK)

```
[
  {
    "id": 1,
    "userId": 1,
    "title": "Order Confirmed",
    "message": "Your order #101 for MacBook Pro has been confirmed",
    "type": "ORDER_CONFIRMED",
    "read": false,
    "createdAt": "2026-03-03T15:30:10"
  }
]
```

🔹 Get Unread Notifications

Method: `GET` **URL:** `/api/notifications/user/{userId}/unread` **Authentication Required:** Yes

1️⃣ What

Returns only unread notifications (where `read = false`). Used for notification badge count on the frontend.

🔹 Mark Notification as Read

Method: `PUT` **URL:** `/api/notifications/{id}/read` **Authentication Required:** Yes

1️⃣ What

Marks a single notification as read. Sets `read = true` in the database.

2️⃣ When

User clicks on a notification in the UI.

🔹 [ADMIN] Get All Audit Logs

Method: `GET` **URL:** `/admin/audit-logs` **Authentication Required:** Yes **Roles Allowed:** ADMIN only

1️⃣ What

Returns all system audit logs ordered by most recent first. Includes user actions (block, unblock), product operations (create, update, delete), and order operations (cancel, status update).

2️⃣ Why

Compliance and security — admins need a full audit trail of all administrative actions performed in the system.

Response Example (200 OK)

```
[
  {
    "id": 1,
    "eventId": "uuid-abc",
    "actor": "admin",
    "action": "CREATE",
    "resourceType": "PRODUCT",
    "resourceId": "42",
    "details": "Created product: MacBook Pro",
    "timestamp": "2026-03-03T15:30:00"
  }
]
```

---

11. Error Handling Strategy

11.1 Global Exception Handler Pattern

Every service has a `GlobalExceptionHandler` (`@RestControllerAdvice`) that catches exceptions and returns consistent JSON error responses.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    // Business logic errors (user not found, duplicate username, etc.)
    @ExceptionHandler(RuntimeException.class)
    → 400 Bad Request + {"message": "...", "status": 400, "timestamp": "..."}

    // Validation errors (@Valid failed)
    @ExceptionHandler(MethodArgumentNotValidException.class)
    → 400 Bad Request + {"errors": {"field": "message"}, "status": 400}

    // Unexpected errors (NullPointerException, DB connection, etc.)
    @ExceptionHandler(Exception.class)
    → 500 Internal Server Error + {"message": "Internal server error"}
}
```

11.2 HTTP Status Code Convention

```
+------+----------------------+-------------------------------------------+
| Code | Meaning              | When Used                                 |
+------+----------------------+-------------------------------------------+
| 200  | OK                   | Successful GET, PUT operations            |
| 201  | Created              | Successful POST (resource created)        |
| 204  | No Content           | Successful DELETE (no body returned)      |
| 400  | Bad Request          | Validation failure, business rule violation|
| 401  | Unauthorized         | Missing or invalid JWT token              |
| 403  | Forbidden            | Valid JWT but insufficient role            |
| 404  | Not Found            | Resource doesn't exist                    |
| 500  | Internal Server Error | Unexpected exception                     |
| 503  | Service Unavailable  | Circuit breaker fallback (service down)   |
+------+----------------------+-------------------------------------------+
```

11.3 Gateway-Level Errors

```
Missing Authorization header:     401 + {"error": "Missing Authorization header"}
Invalid Bearer format:             401 + {"error": "Invalid Authorization header"}
Expired JWT token:                 401 + {"error": "Token expired or invalid"}
JWT signature mismatch:            401 + {"error": "Token validation failed"}
Service unavailable (CB open):     503 + {fallback response from FallbackController}
```

---

12. API Design Patterns

12.1 Two-Controller Pattern

```
Every service with admin operations has TWO controllers:
  /api/resource/*     → UserController (any authenticated user)
  /admin/resource/*   → AdminController (ADMIN role required)

WHY?
  1. Clean separation of user vs admin endpoints
  2. @PreAuthorize applied at class level on AdminController
  3. Different SecurityConfig rules per path prefix
  4. Admin operations include Kafka events + audit logging
```

12.2 Header-Based Identity Pattern

```
Gateway extracts JWT claims → injects as HTTP headers:
  X-User-Name: john       (from JWT sub claim)
  X-User-Role: ROLE_USER  (from JWT role claim)

Downstream services read headers:
  @RequestHeader("X-User-Name") String username

WHY NOT parse JWT in every service?
  1. Single point of validation (Gateway)
  2. Simpler service code (just read a header)
```

```
  3. Consistent — all services get identity the same way
  4. Testable — easy to mock headers in unit tests
```

12.3 Fire-and-Forget Kafka Pattern

```
// AdminProductController.java
CompletableFuture.runAsync(() -> {   // Runs in ForkJoinPool thread
    publishAudit(username, "CREATE", id, "Created product");
    publishProductEvent(id, "CREATED", dto);
});
return ResponseEntity.status(201).body(created);  // Returns IMMEDIATELY

WHY?
  HTTP response should NEVER wait for Kafka.
  If Kafka is slow (5s): product is already in DB, user already got 201.
  If Kafka fails: log warning, but product creation succeeded.
  Audit is best-effort, not transactional.
```

12.4 Soft Delete Pattern

```
Products are NOT deleted from database:
  product.setActive(false)   // Soft delete
  productRepository.save(product)

WHY?
  1. Orders reference productId — hard delete breaks FK integrity
  2. Audit trail — "who deleted what" requires the record to exist
  3. Recovery — admin can re-activate accidentally deleted products
```

12.5 DTO Pattern

```
NEVER expose JPA entities in API responses.

Entity (internal):   Product { id, name, price, active, createdAt, updatedAt, @Version }
DTO (external):      ProductDto { id, name, price, description, category }

WHY?
  1. Security — don't expose @Version, password hash, internal flags
  2. Flexibility — API shape can differ from DB schema
  3. Backward compatibility — DB schema changes don't break API contract
```

---

13. Interview Questions

Q1: Why do you have separate /api/ and /admin/ paths? Separation of concerns. /api/ endpoints are for regular authenticated users. /admin/ endpoints require ADMIN role. Admin endpoints include additional Kafka event publishing and audit logging that user endpoints don't need. Security rules are cleaner: `requestMatchers("/admin/**").hasRole("ADMIN")` .

Q2: Why doesn't the Payment Service have more REST endpoints? Payment Service is event-driven. Payments are triggered by Kafka events (inventory-updated), not REST calls. Users never directly call "pay now" — the system automatically processes payment after inventory reservation. The single GET endpoint is for querying payment records after the fact.

Q3: How is the user identity propagated without session state? JWT is stateless. Gateway validates the token once, extracts username and role, and injects them as X-User-Name and X-User-Role headers. Downstream services read these headers — no session store, no shared state. Any service instance can handle any request.

Q4: Why is the order status initially PENDING and not CONFIRMED? Because order confirmation requires inventory check and payment processing — both happen asynchronously via Kafka. The user gets an immediate response (201 Created, status: PENDING) while the saga proceeds in the background. The frontend polls `GET /api/orders/{id}` to track status changes.

Q5: Why is Kafka event publishing wrapped in CompletableFuture.runAsync()? HTTP response latency. If Kafka takes 500ms to acknowledge, every admin product creation takes 500ms longer. By using fire-and-forget async, the HTTP response returns in ~50ms (DB write only). Kafka publishing happens in a background thread. Trade-off: if Kafka fails, the audit log is lost — acceptable for audit (best-effort) but not for order events (uses synchronous send).

Q6: Why does Inventory Service return entity objects directly while other services use DTOs? Design choice. Inventory data has no sensitive fields to hide — quantity, reservedQuantity, availableQuantity are all public. Adding a DTO layer would add complexity without security benefit. However, in a strict enterprise codebase, DTOs would be used for all services for consistency.

End of Document

This guide documents all 43 API endpoints across 8 microservices with request/response examples, architectural reasoning, security analysis, and design patterns.